

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 5

Duration: 1 Hour
Total Marks:50

Annexure A

Scenario Based Assignment

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

1. Scenario: AI-Powered Emergency Medicine Delivery Drones

Scenario: A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. Roads and routes may suddenly become inaccessible, weather affects travel cost, the drone has only a heuristic estimate such as straight-line distance, and it receives partial real-time updates about blocked paths and changing movement costs. The drone must quickly choose routes with minimum delay and risk.

i. Greedy Best-First Search

Greedy Best-First Search selects the node that appears closest to the destination using only the heuristic value $h(n)$. For the drone, it would prefer routes that seem geographically closest to the medicine delivery point.

Its main strength is speed because it does not spend much time comparing the complete cost of alternative routes. This is useful in emergencies. However, it can select a dangerous or expensive route because it ignores the actual cost already travelled. A route that looks short may contain flooding, strong winds, obstacles, or high energy consumption. Therefore, Greedy Best-First Search is fast but neither complete nor optimal in many real-world situations.

ii. A* Search

A* evaluates each node using $f(n) = g(n) + h(n)$, where $g(n)$ is the actual accumulated travel cost and $h(n)$ is the estimated remaining cost to the destination. The travel cost can include distance, delay, battery usage, weather risk, and terrain difficulty.

Because A* considers both the cost already incurred and the estimated future cost, it makes a more balanced decision than Greedy Best-First Search. With an admissible and consistent heuristic, A* can find an optimal path when the environment is static.

iii. Impact of Heuristic Accuracy

A more accurate heuristic guides both algorithms toward promising routes and reduces unnecessary exploration. Greedy Best-First Search depends heavily on $h(n)$, so a poor heuristic can easily lead it toward an inefficient or unsafe route.

A* is more robust because it also uses $g(n)$. If $h(n)$ underestimates the true remaining cost, A* can preserve optimality. If the heuristic greatly overestimates costs, A* may become faster but can lose its optimality guarantee.

iv. Effect of Dynamic Changes

Blocked routes and changing weather can make a previously selected path invalid. Both algorithms may need to replan when new information arrives. Standard Greedy Best-First Search and A* are designed mainly for static graphs, so repeated full searches can be expensive.

For a real drone, the map and edge costs should be updated whenever new sensor or communication data arrives. The system should then repair or recompute the route rather than blindly following the old plan.

v. Recommendation

A* is more suitable than Greedy Best-First Search when route quality and safety are important because it balances actual cost and estimated remaining cost. For a highly dynamic environment, an incremental variant such as D* Lite or Lifelong Planning A* is even more suitable because it can efficiently replan when paths become blocked.

The recommended system should use a risk-aware cost function, real-time updates, and an admissible heuristic. This provides a good balance between fast response, near-optimal/optimal routing, adaptability, and safety.

2. Scenario: Smart City Traffic Signal Optimization

Scenario: A smart city uses AI to optimize traffic signal timings across hundreds of intersections. The objectives are to minimize total waiting time, fuel consumption, and traffic congestion while continuously adapting to changing traffic flow. The search space is extremely large and may contain many locally optimal solutions.

Problem Formulation and Why Traditional Search Is Inefficient

Let each candidate state represent a complete set of signal timings for all intersections. Decision variables may include green time, red time, cycle length, phase order, and offsets between nearby signals. An objective function can minimize a weighted combination such as total waiting time + congestion + fuel consumption/emissions, subject to safety and minimum/maximum timing constraints.

With hundreds of intersections and many timing choices at each one, the number of possible configurations grows exponentially. Exhaustively evaluating every configuration is therefore computationally impractical, especially when traffic conditions change continuously.

i. Hill Climbing

Hill Climbing starts with an initial traffic-signal configuration and evaluates nearby configurations. It repeatedly moves to a neighboring state that improves the objective value, for example by slightly changing the green time at one or more intersections.

It is simple, memory-efficient, and can quickly improve a solution. However, it uses local information and may stop at a local optimum even though a much better configuration exists elsewhere in the search space.

ii. Local Maxima, Plateaus and Ridges

A local maximum/minimum occurs when the current solution is better than all nearby solutions but is not globally best. In traffic control, one junction may be optimized while causing congestion in surrounding roads.

A plateau is a region where many neighboring configurations have nearly the same evaluation value, giving the algorithm no clear direction. A ridge occurs when improvement requires a coordinated sequence of changes that simple local moves cannot easily follow. These situations can cause Hill Climbing to stop early or progress very slowly.

iii. Simulated Annealing or Genetic Algorithms

Simulated Annealing sometimes accepts a worse move, especially early in the search. This allows it to escape local optima and explore other regions. As the temperature decreases, it becomes more selective and converges toward a good solution.

Genetic Algorithms maintain a population of signal-timing solutions. Selection keeps better candidates, crossover combines useful features from different solutions, and mutation introduces diversity. This makes GA effective for large, complex and multi-objective search spaces, although evaluating many candidates can require substantial computation.

iv. Recommended Approach

For a large smart-city network, a Genetic Algorithm is a strong choice because it can optimize many timing variables simultaneously, handle multiple objectives and constraints, and search beyond a single local neighborhood. It can also be parallelized across processors.

For continuous real-time operation, a hybrid approach is even better: use GA periodically to obtain high-quality global configurations and use fast local or adaptive optimization between major updates. This improves scalability and adaptability while keeping response time practical.

3. Scenario: Autonomous Mars Rover Exploration

Scenario: An autonomous rover explores unknown Martian terrain. It must collect samples, avoid hazards such as craters and rocks, operate with limited sensing, and make decisions without a complete map. Its knowledge is updated as it explores, so navigation is a real-time decision-making problem.

i. Why an Online Search Agent Is Required

Offline search assumes that the environment, actions, and relevant map information are known before execution. The rover does not have complete information about the terrain, so it cannot construct a complete plan in advance.

An online search agent interleaves planning and action. It observes the current surroundings, chooses an action, moves, receives new sensor information, and plans again. This is suitable for unknown terrain and communication delays.

ii. Partial Observability and Dynamic Environment Challenges

The rover can observe only a limited area around itself, and sensors may be affected by distance, dust, lighting, or terrain. Therefore, it may not know whether a route is safe until it approaches it.

Conditions can also change: loose soil may reduce mobility, a path may become unsafe, and energy availability may vary. The rover must make decisions under uncertainty while avoiding irreversible or dangerous actions.

iii. Building and Updating an Internal Model

The rover stores observations in an internal world model or map. Each observation can update information about free space, obstacles, terrain difficulty, scientific targets, estimated movement cost, and confidence in the data.

After every movement, sensor readings are integrated with previous knowledge. Unknown areas become known, edge costs may be changed, and unsafe regions can be marked as blocked. This continuously updated model supports future planning and prevents repeated mistakes.

iv. Online Search vs. Uninformed and Informed Search

Uninformed strategies such as BFS or Uniform-Cost Search explore using little or no goal-specific knowledge. They may require many expansions and are inefficient when the environment is very large or partly unknown.

Informed strategies use heuristic knowledge to guide exploration toward useful locations. A* is efficient when a sufficiently accurate map is available, but online search is necessary when the map itself must be discovered during execution. Thus, online search describes how planning and action are interleaved, while informed heuristics can still be used inside the online agent.

v. Most Suitable Approach

A model-based online search agent with incremental heuristic planning is the most suitable. It should continuously update its internal map and replan using algorithms such as D* Lite or another incremental A* method.

Safety constraints should be treated as high priority: uncertain high-risk terrain should receive a large cost, the rover should preserve sufficient energy for recovery, and it should choose information-gathering actions when uncertainty is too high. This approach provides adaptability, efficient exploration, and safer real-time decisions.

4. Scenario: University Exam Timetable Generation

Scenario: A university wants an AI system to automatically generate exam timetables. It must prevent overlapping exams for students, respect limited exam halls, enforce subject ordering requirements, consider faculty availability, and provide special accommodations. The problem becomes more complex as courses and students increase.

CSP Formulation

The problem can be represented as a Constraint Satisfaction Problem (CSP) consisting of variables, domains, and constraints.

i. Variables, Domains and Constraints

Variables: each exam/course can be treated as a variable, for example X_1 = Mathematics exam, X_2 = AI exam, and so on. Additional variables may represent hall assignments.

Domains: the possible values for each exam include available date/time slots and suitable halls.

Constraints: two exams sharing students cannot occur at the same time; a hall cannot host incompatible simultaneous exams beyond capacity; faculty must be available; prerequisite or ordering requirements must be respected; and students requiring accommodations must receive suitable rooms or timing. These are hard constraints, while preferences such as spreading exams may be modeled as soft constraints.

ii. Backtracking Search

Backtracking assigns a value to one variable at a time. For example, it assigns a time slot and hall to the first exam, then continues with the next exam. After each assignment, the algorithm checks whether any constraint is violated.

If a later exam has no valid assignment, the algorithm returns to an earlier decision, changes it, and tries another value. This continues until a complete consistent timetable is found or all possibilities are exhausted.

iii. Constraint Propagation / Forward Checking

Forward checking improves backtracking by removing inconsistent values from the domains of unassigned variables immediately after an assignment. For example, if an exam is scheduled on Monday at 10 AM, the same slot can be removed from exams that share students or require the same unavailable hall.

Constraint propagation detects failure earlier and reduces the number of branches explored. Heuristics such as Minimum Remaining Values (MRV), Degree Heuristic, and Least Constraining Value can further improve performance.

iv. Real-World Challenges and Best Scalable Strategy

Challenges include thousands of students, changing faculty availability, hall capacities, special accommodations, late course changes, and conflicting soft preferences. A simple backtracking method may become slow as the problem grows.

A strong scalable strategy is backtracking with constraint propagation, MRV, degree heuristic, and least-constraining-value ordering. For very large institutions, a modern CSP/constraint-programming or SAT/CP-SAT solver can handle hard constraints and optimize soft preferences more effectively.

5. Scenario: Strategic Game AI for Real-Time Decision Making

Scenario: A gaming company is developing an AI opponent for a real-time strategy game. The AI competes against human players, must make decisions within strict time limits, and faces a very large game tree with many possible moves. Evaluating complete game trees is computationally expensive.

i. Minimax Model

Minimax models the game as an adversarial search tree. MAX represents the AI player and tries to maximize the utility value, while MIN represents the opponent and tries to minimize it.

The algorithm expands possible moves until terminal states or a chosen depth limit. Values are then propagated upward: MAX selects the highest value and MIN selects the lowest. This assumes that both players make rational choices.

ii. Computational Challenges

If the average branching factor is b and the search depth is d , basic Minimax has time complexity $O(b^d)$. In a real-time strategy game, both b and d can be very large, causing an enormous number of states.

Memory use, repeated states, limited CPU time, and the need to react quickly make full-depth Minimax impractical. Therefore, the AI usually searches only part of the tree and estimates the value of non-terminal positions.

iii. Alpha-Beta Pruning

Alpha-Beta pruning produces the same decision as Minimax while avoiding branches that cannot influence the final result. Alpha represents the best value MAX can guarantee so far, while beta represents the best value MIN can guarantee.

When alpha becomes greater than or equal to beta, the remaining children of that node can be pruned because the opponent already has a better alternative. Good move ordering greatly increases pruning. In the best case, Alpha-Beta can reduce the effective work from about $O(b^d)$ toward $O(b^{(d/2)})$, allowing deeper search in the same time.

iv. Evaluation Function

At the depth limit, an evaluation function estimates how favorable a state is. A possible function is: $\text{Score} = 4(\text{Material Advantage}) + 3(\text{Strategic Position}) + 2(\text{Resource Control}) + 2(\text{Unit Safety}) + 1(\text{Mobility}) - 4(\text{Immediate Threat})$.

Material advantage measures the relative strength of units; strategic position measures control of important areas; resource control estimates future production; unit safety penalizes exposed units; mobility rewards useful available actions; and immediate threat heavily penalizes states close to defeat. The weights can be tuned using game data and testing.

v. Recommended Real-Time Strategy

Use depth-limited Minimax with Alpha-Beta pruning, strong move ordering, an evaluation function, and iterative deepening. Iterative deepening first searches shallow levels and then increases depth while time remains, ensuring that a usable move is always available before the deadline.

Transposition tables can store previously evaluated states, and selective search can focus computation on tactically important moves. This combination provides a practical balance between decision quality, computational cost, and real-time performance.

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand